How the FoC services talk to each other and how each one is packaged into a container

A code-grounded teaching guide to the CS3219 P32 repository

Prepared for Kavish · 19 October 2026 · repository snapshot pinned in Appendix B

This guide explains the system that is actually in the repository: which programs run, which messages they exchange over Google Cloud Pub/Sub, what each message looks like, what each handler writes to which database, and how every program is built into a Docker image and started by Docker Compose (locally) or Cloud Run (in Google Cloud). It starts from an everyday mental model, introduces the vocabulary only when it is needed, and then follows real requests end to end. Where something is configured but has no evidence of having run, or where a link in a flow is missing, the text says so. File paths and line numbers are collected in Appendix B rather than in the running text, so that the explanations stay readable.

Contents

1 A familiar starting point: phone calls and mailing lists **2**

2 Vocabulary, tied to this codebase **3**

3 The cast: every program that actually runs **5**

4 Synchronous HTTP: who calls whom, and with what credentials **6**
- 4.1 The browser talks only to Nuxt (and one WebSocket) . . . 7
- 4.2 Each backend verifies the token itself . . . 7
- 4.3 The notification WebSocket uses a short-lived ticket . . . 7
- 4.4 Internal routes . . . 8

5 The asynchronous map: topics, publishers and subscriptions **8**
- 5.1 `user-events` . . . 8
- 5.2 `supplier-events` . . . 9
- 5.3 `order-events` . . . 10
- 5.4 `credit-commands` . . . 10
- 5.5 `credit-events` . . . 11
- 5.6 `order-commands` . . . 11

6 How a delivery is handled in code: push, pull and acknowledgements **12**
- 6.1 What a push request looks like . . . 12
- 6.2 The HTTP status is the acknowledgement . . . 12
- 6.3 Pull delivery in notification-service . . . 13

7 The outbox pattern as implemented here **13**
- 7.1 Writing: the change and the message row commit together . . . 13
- 7.2 Relaying: at least once, in order per key . . . 14
- 7.3 What this does and does not guarantee . . . 15

8 Walkthrough A: a sign-up replicates the user to four services **15**

9 Walkthrough B: an order from creation to settlement **17**
- 9.1 State machines first . . . 17
- 9.2 Phase 1: creating and opening the order . . . 18
- 9.3 Phase 2: doing the errand and paying the courier . . . 19

10 Walkthrough C: refusals, timeouts, compensation and duplicates **21**
10.1 Not enough credits: the order is rejected . . . 21
10.2 Credit does not answer in time: abort and release . . . 21
10.3 Cancelled or expired orders: release . . . 21
10.4 Waiting for prerequisites through retries . . . 21
10.5 Duplicates, layer by layer . . . 22
10.6 What is not handled automatically . . . 22

11 Walkthrough D: a supplier edit reaches order-service **22**

12 Local emulator vs Google Cloud transport **23**
12.1 Locally: services set up their own topics and subscriptions . . . 23
12.2 In Google Cloud: `deploy.sh` creates the transport . . . 23
12.3 Configured versus actually exercised . . . 24

13 Containers from first principles **24**

14 Each Dockerfile, explained **25**
14.1 The three-stage pattern: supplier, order, notification, saga . . . 25
14.2 credit-service: two stages on Alpine . . . 26
14.3 user-service: one stage, TypeScript at runtime . . . 26
14.4 frontend-service: build, then ship only the output . . . 27
14.5 Images used as-is . . . 27

15 Compose: the running system **27**
15.1 One network, names as addresses . . . 27
15.2 Startup order: `depends_on` is not the same as "ready" . . . 28
15.3 Volumes and persistence . . . 29
15.4 Environment and secrets . . . 29

16 `compose.dev.yaml`: hot reloading **29**

17 Databases: ownership, persistence and migrations **30**

18 Cloud Build and Cloud Run packaging **30**

19 Gaps, missing links and things to keep in mind **31**

20 Notes for the upcoming event bus tap **31**

A Running the system locally for a demo **32**

B Reference: snapshot and provenance **32**

1 A familiar starting point: phone calls and mailing lists

Services in this project communicate in two very different ways, and almost every design detail later in the guide follows from the difference.

A phone call (synchronous HTTP). When the browser asks the Nuxt frontend for the list of orders, the frontend's server makes an HTTP request to `order-service` and *waits* for the answer. Both parties must be available at the same moment, and the caller learns immediately whether it worked. That is how every user-facing read and write in this project starts.

A mailing list with personal mailboxes (asynchronous Pub/Sub). When a student signs up, `user-service` does not phone the supplier, order and credit services one by one. It posts a single letter, a *message*, to a mailing list called `user-events`. Every service that cares about users has its own mailbox on that list, called a *subscription*. The post office (Pub/Sub) puts a copy of the letter

in every mailbox and keeps it there until the owner says "got it", which is called *acknowledging*. If a mailbox owner is away, the letter waits; if the owner reads it but does not confirm, the post office delivers it again later.

Two consequences follow, and both show up all over the code:

- The sender does not know when, or whether, each reader has acted on the letter. A freshly registered user can briefly be unknown to `order-service`. The code handles this lag explicitly, for example with a "your account is still being set up" response.
- The post office may deliver the same letter more than once. Every reader in this repository therefore checks whether it has already applied a letter before changing its database. That property is called *idempotency*.

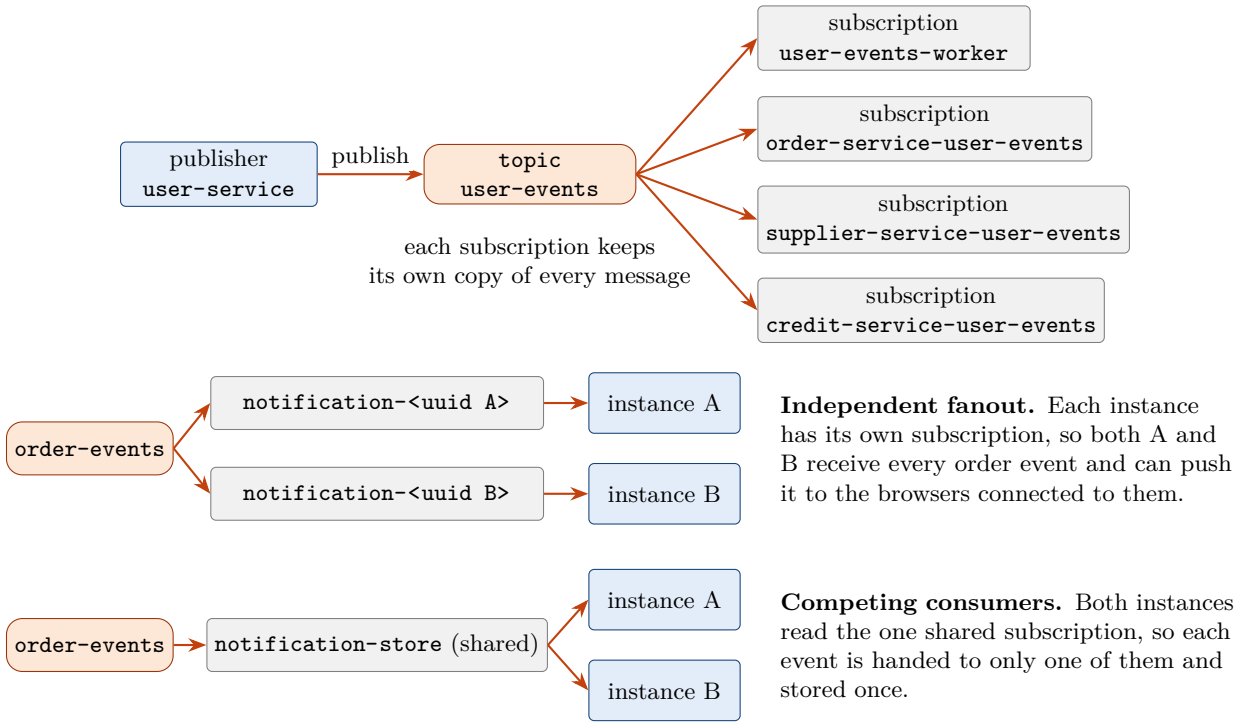


Figure 1: Top: one topic, four mailboxes (subscriptions), each a separate copy. Bottom: the notification service uses both patterns on one topic: a private subscription per running instance, plus one shared subscription that the instances split between them.

> **Why it matters.** Once you see a topic as "a mailing list" and a subscription as "one reader's mailbox", the rest of the repository becomes predictable: anyone who wants every message creates a new subscription; anyone who wants to share the work joins an existing one; and anyone who reads mail must cope with letters that arrive late or twice.

2 Vocabulary, tied to this codebase

Topic. A named channel that publishers send messages to. The repository uses six: `user-events`, `supplier-events`, `order-events`, `order-commands`, `credit-commands` and `credit-events`. A topic by itself stores nothing for anyone; only subscriptions hold messages.

Subscription. A named, durable queue attached to one topic. Pub/Sub only puts a message into the subscriptions that *already exist* when the message is published. That rule explains several startup dependencies in `compose.yaml` (Section 15.2).

Message. A message has four parts that matter here. The *data* is the body; every service in this repository puts a JSON document there. The *attributes* are small string key–value pairs outside the

body, such as `type=order.created`, which Pub/Sub can filter on. The *ordering key* groups related messages, for example all messages about one order. The *message ID* is assigned by Pub/Sub. The services do not rely on Pub/Sub's message ID; they put their own identifiers (`eventId`, `commandId`, `requestId`) inside the data.

Push subscription. Pub/Sub delivers each message by making an HTTP `POST` to an endpoint URL owned by the consumer. A success status (this code answers `204` or `200`) acknowledges the message. Any other status, or no answer within the acknowledgement deadline, counts as a failure, and Pub/Sub retries later with backoff. Every consumer in the repository except the notification service uses push.

Pull subscription. The consumer's client library connects to Pub/Sub and receives messages; the code then calls `message.ack()` or `message.nack()` itself. Only `notification-service` uses pull. A pull consumer needs no public endpoint and can be a long-lived process that also holds WebSocket connections.

At-least-once delivery, retries and duplicates. Pub/Sub may deliver a message again even after the handler succeeded, for example if the acknowledgement was lost. Publishers in this repository can also send a message twice (Section 7). Nothing in the code relies on exactly-once delivery.

Idempotency. A handler is idempotent if applying the same message twice leaves the same state as applying it once. The repository uses four techniques, and it is worth learning to spot them:

- *Version comparison*: "only overwrite my copy if this message's `version` is newer" (user copies in user-, supplier- and order-service).
- *Natural unique keys*: insert with "on conflict do nothing" on a key such as (supplier id, version) or a notification id derived from the event id.
- *A processed-messages table*: the saga orchestrator records each `eventId` it has handled and skips repeats.
- *Request identifiers*: credit-service stores every command under its `requestId` and returns the stored result when the same request arrives again; order-service refuses to apply the same `commandId` twice.

Ordering key. When a subscription has message ordering enabled, Pub/Sub delivers messages that share an ordering key in the order they were published, one at a time. If a publish fails, the client library pauses that key until the code calls `resumePublishing(key)`, which every publisher here does in its error path.

Filter. A subscription can be created with a filter on attributes, so unwanted messages never reach it. The saga's order-events subscription accepts only four order event types; the notification subscriptions accept any type starting with `order.`.

Independent fanout vs competing consumers. Several subscriptions on one topic each receive every message (fanout). Several processes reading the *same* subscription share its messages, so each message goes to one of them (competing consumers). Figure 1 shows both.

Events vs commands. An *event* reports a fact that has already happened (`order.created`, `credit.reserved`); the publisher does not care who listens. A *command* asks one specific service to do something (`credit.reserve`, `OpenOrder`). In this repository, commands travel on their own topics (`order-commands`, `credit-commands`). In the normal workflow only the saga orchestrator sends them. The one exception is an operator test tool outside that workflow: credit-service's `npm run command:publish`, which publishes a single command from a JSON file straight to `credit-commands`.

Transactional outbox. Instead of publishing directly, a service writes the message as a row in an "outbox" table *inside the same database transaction* as the business change, and a separate relay later publishes unpublished rows. The business change and the record of the message therefore commit or roll back together. Section 7 explains exactly what this does and does not guarantee here.

Saga and orchestrator. A saga is a multi-step business process spread across services, where each step is a local transaction and a failure later is handled by a *compensating* step (for example releasing reserved credits) rather than a global rollback. Here a dedicated service, `saga-orchestrator`, drives the steps by listening to events and sending commands.

Projection (local copy). A table that a service keeps up to date from events so that it can answer questions without calling another service. Order-service, for example, keeps its own `users` and `supplier_versions` tables.

3 The cast: every program that actually runs

The repository has one folder per service. Several folders produce more than one running program: a service's image is often started a second time with a different command to run a worker. The list below follows `compose.yaml`, which is the most complete description of what runs together.

`frontend-service` (Nuxt 4)
The only thing the browser loads. Its Vue pages call the Nuxt server's own `/api/...` routes, and those server routes call the backends with the user's token. It owns no database. Inside Compose it listens on port 3000, published on the host as 3001.

`user-service` and `user-event-worker`
One image, two programs. The API (port 3333) handles registration, login, profiles and administrator user management, and signs the JWTs that every other service verifies. It does *not* write users to its database itself: it publishes `user.created`, `user.updated` or `user.deleted` to `user-events`. The worker (port 3334, not published) receives those events by push and writes them to the `users` table that the API reads. Database: `user-db`.

`supplier-service`, `supplier-event-worker` and `supplier-ticker`
The API (port 3000) serves suppliers, supplier types and buildings, and administrators can change them. Each change also writes an outbox row, which is published to `supplier-events`. The worker (port 3004 inside its container, not published) does two jobs: it keeps a local `users` copy from `user-events`, which the API uses for authorization, and it exposes `POST /outbox/publish`, which publishes any outbox rows the API missed. The ticker is a tiny `curl` container that calls that sweep every 60 seconds. Database: `supplier-db`.

`order-service` and `order-ticker`
The order API (port 3000, host 3002). It keeps local copies of users and of every supplier version, records each order state change as a row in `order_events`, which doubles as its outbox, and publishes those rows to `order-events`. It also receives saga commands on `order-commands`. The ticker calls two internal routes every 60 seconds: one expires overdue open orders and one republishes leftover outbox rows. Database: `order-db`.

`saga-orchestrator` and `saga-ticker`
Has no user-facing API and no host port. It reacts to four order event types and to credit results, keeps one saga row per order, and sends commands to `credit-commands` and `order-commands` through its own outbox. Its ticker aborts reservations that credit did not answer in time and republishes leftover commands. Database: `saga-db`.

`credit-service`, `credit-publisher` and `credit-pubsub-provision`
The API (port 3000, host 3004) answers `GET /api/v1/credits/me` for the browser and receives `user-events` and `credit-commands` by push. Results go to a `credit_outbox` table. The publisher is a separate, always-running loop that publishes those rows to `credit-events` about once a second. The provision container runs once at startup and creates credit's topics and push subscriptions. Database: `credit-db`.

`notification-service`
Pulls `order-events`, pushes live updates to browsers over WebSockets on `/ws`, and stores per-user

notifications. Port 3000, host 3003; the browser connects to it directly for the WebSocket. Database: `notification-db`.

Infrastructure
`pubsub-emulator` (Google's local Pub/Sub emulator on port 8085, not published to the host); six PostgreSQL 17 containers, one per database (`supplier-db`, `user-db`, `order-db`, `notification-db`, `credit-db`, `saga-db`); one-shot migration containers (`*-db-migrate`); and `supplier-db-tools`, which only starts when the `tools` profile is requested.

Program	Listens in container	Host port	Database it owns
frontend-service	3000	3001	none
user-service / its event worker	3333 / 3334	3333 / none	user-db (users)
supplier-service / its event worker	3000 / 3004	3000 / none	supplier-db (suppliers)
order-service	3000	3002	order-db (orders)
notification-service	3000	3003	notification-db (notifications)
credit-service / credit-publisher	3000 / 3000 (health)	3004 / none	credit-db (credits)
saga-orchestrator	3000	none	saga-db (sagas)
pubsub-emulator	8085	none	in memory

Table 1: Ports and database ownership in `compose.yaml`. PostgreSQL containers are also published on the loopback interface only: 5433 supplier, 5434 user, 5435 order, 5436 notification, 5438 credit, 5439 saga.

Why it matters. "One service, one database" is enforced physically here: each database is a separate container that only its own service is configured to reach. That is why services need events to learn about each other's data, and why the event payloads carry whole snapshots rather than just IDs.

4 Synchronous HTTP: who calls whom, and with what credentials

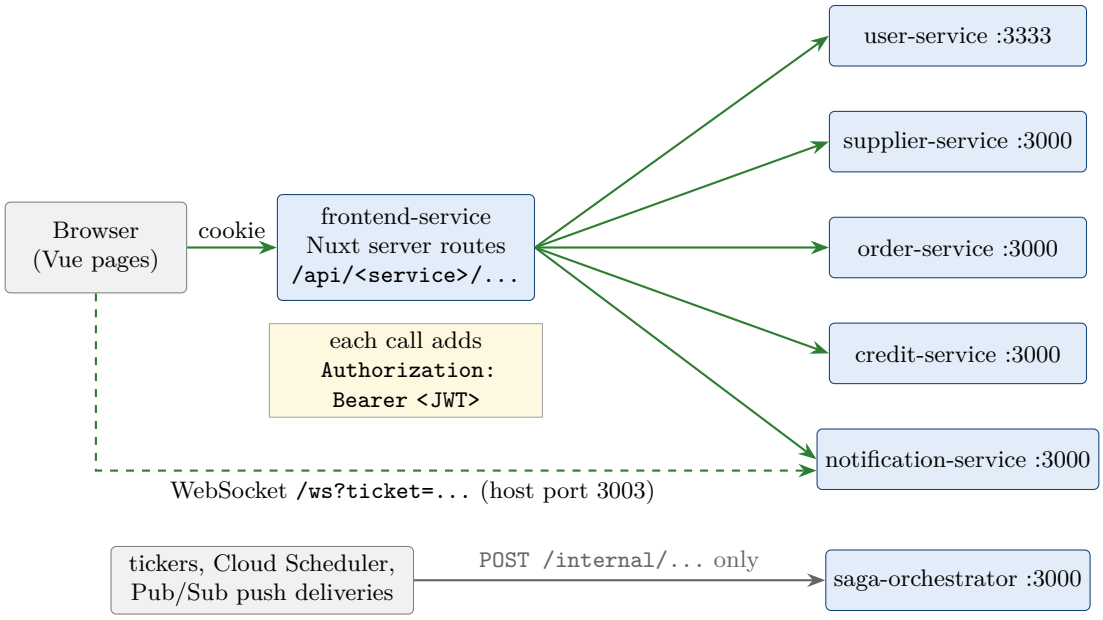


Figure 2: All user-initiated HTTP goes browser → Nuxt server → backend. No backend calls another backend over HTTP. The only other HTTP callers are infrastructure: the tickers or Cloud Scheduler, and Pub/Sub push deliveries.

4.1 The browser talks only to Nuxt (and one WebSocket)

The Vue pages never hold the JWT. When a user logs in, the Nuxt server route forwards the email and password to `user-service` `POST /login`, receives a token, and stores it in an `httpOnly` cookie named `user_access_token`:

```
// frontend-service/server/api/user-service/login.post.ts (excerpt)
setCookie(event, "user_access_token", response.token, {
  httpOnly: true, sameSite: "lax",
  secure: process.env.NODE_ENV === "production", path: "/",
});
```

Every later Nuxt server route reads the cookie and calls the backend with `Authorization: Bearer <token>`, using a base URL from runtime configuration. In Compose those URLs use service names, for example `NUXT_ORDER_SERVICE_BASE_URL=http://order-service:3000` (Section 15.1). The clients use a 5-second timeout and no automatic retry. The order, supplier and user clients pass through the statuses the user should see (400, 401, 403, 404, 409 and 503); anything else becomes a 502 with a friendly message.

4.2 Each backend verifies the token itself

The token is an HS256 JWT signed by `user-service` with `JWT_SECRET`; its `sub` claim is the numeric user ID and `exp` the expiry. Supplier-, order-, credit- and notification-service each contain their own copy of the verifier and receive the same `JWT_SECRET`. They check the signature and expiry locally, without calling `user-service`.

Supplier-service and order-service then look the caller up in their *local copy* of users, which is filled from `user-events`:

```
// order-service/src/auth.ts, requireUser (excerpt)
const [user] = await db.select({ deletedAt: users.deletedAt })
  .from(users).where(eq(users.id, userId));
if (!user) {
  throw new HttpError(503, "Your account is still being set up. Please try again shortly.
    ",
    { "retry-after": "2" });
}
if (user.deletedAt) throw new HttpError(401, "A valid bearer token is required");
```

This is the replication lag from Section 1 made visible. Just after registration the token is valid, but the `user.created` event may not have reached order-service yet, so the request gets a retryable 503 rather than a confusing 401. Supplier-service does the same and also reads the caller's `role` from its copy to allow administrator actions. Credit-service checks only the token, then answers 503 `ACCOUNT_PENDING` until its own account row exists.

4.3 The notification WebSocket uses a short-lived ticket

Because the browser cannot read the httpOnly cookie, it cannot put the JWT in a WebSocket URL. Instead the page asks the Nuxt route `POST /api/notification-service/ticket`, which calls notification-service `POST /tickets` with the JWT. Notification-service returns a signed, single-use ticket that is valid for 30 seconds by default. The browser then opens `ws://127.0.0.1:3003/ws?ticket=...` directly; in Cloud the scheme is `wss://`. The server checks the `Origin` header against `ALLOWED_ORIGINS`, redeems the ticket, and closes the socket when the underlying JWT would expire.

4.4 Internal routes

The saga orchestrator and the `/internal/...` routes of order- and credit-service are not meant for users. They accept a request only if it carries either a Google-signed OIDC token from an allowed service account (Cloud) or, locally, a shared secret in the `x-internal-secret` header or a `token` query parameter (Section 12).

> **Why it matters.** Keeping synchronous calls to "frontend → one backend" means a user request never fails because some *other* backend is down. The price is eventual consistency: the other services learn about changes through events, a little later.

5 The asynchronous map: topics, publishers and subscriptions

Figures 3 and 4 show every topic and every subscription that the code or the deployment script creates. The arrows into a topic are publishes; the arrows out are subscriptions, labelled with the subscription name and delivery style.

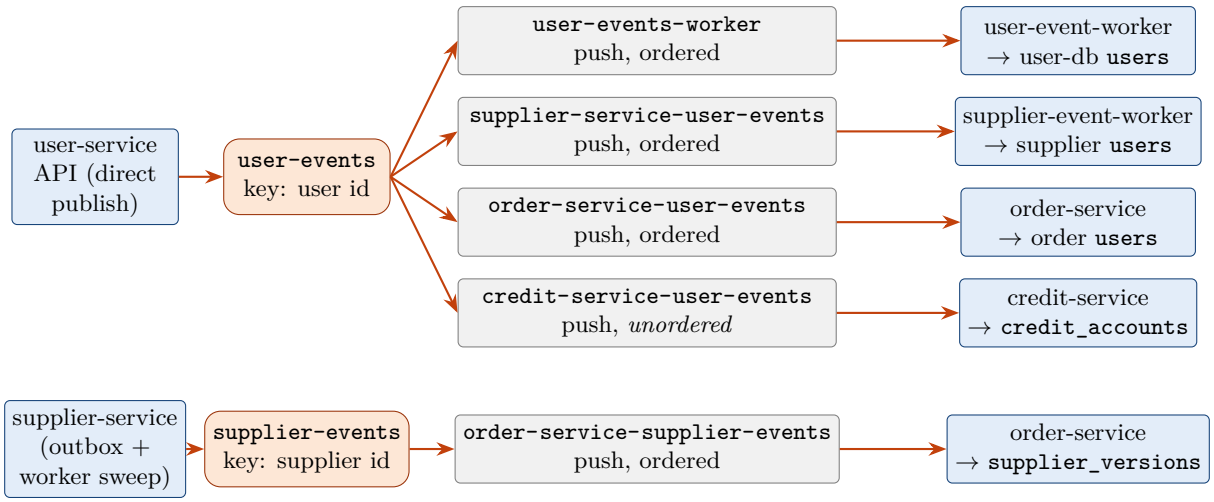


Figure 3: Data replication topics. One user event fans out to four independent subscriptions; one supplier event has a single consumer.

The following subsections describe each topic in the same order: who publishes and how, then the ordering key and attributes, then the subscriptions, and finally the real payload shape with the meaning of every field. Values in the examples are illustrative, but the field sets come directly from the code that builds them.

5.1 `user-events`

Publisher. `user-service`, directly from the request handler, with no outbox. Registration, administrator create/update/delete, self-service profile and password changes, and first-administrator bootstrap all call `publishUserEvent`. The function reserves the next value of the PostgreSQL sequence `user_event_sequence` as the event's `version`, generates a random `eventId`, and publishes with the user ID as ordering key. It sets no attributes.

Subscriptions (all push). `user-events-worker` feeds `user-event-worker`, `supplier-service-user-events` feeds `supplier-event-worker`, `order-service-user-events` feeds `order-service`, and `credit-service-user-events` feeds `credit-service`.

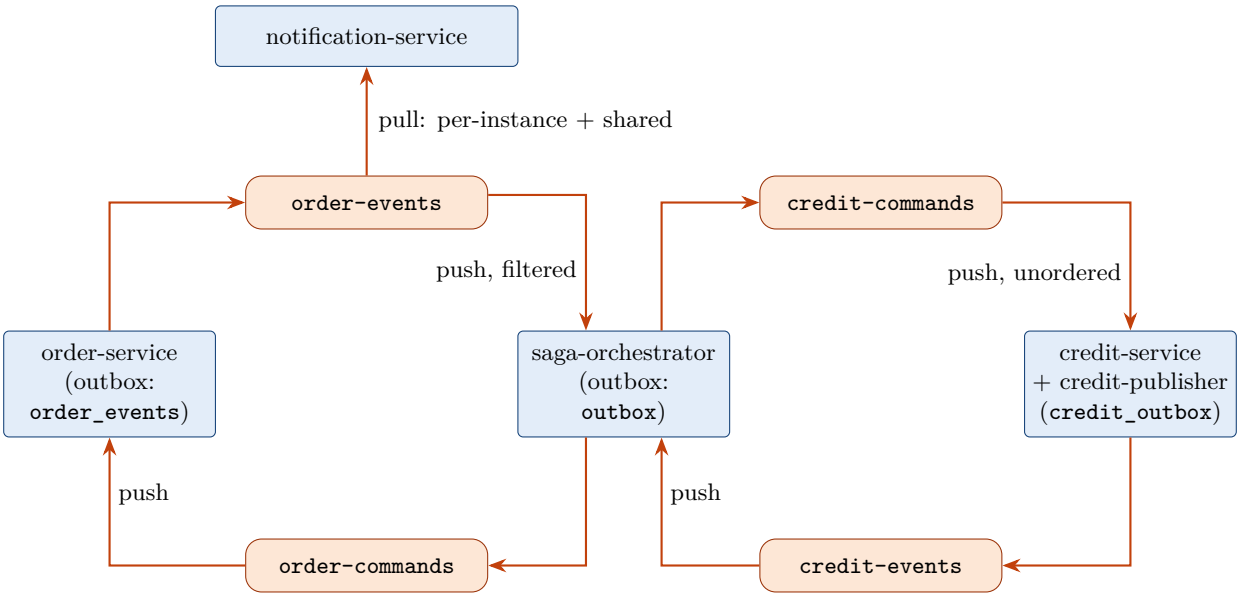


Figure 4: Order workflow topics. Events flow up and to the right; commands come back from the saga. Subscription names: `saga-orchestrator-order-events`, `credit-service-commands`, `saga-orchestrator-credit-events`, `order-service-order-commands`.

```
{ "type": "user.created",
  "user": { "id": 7, "name": "Ada Tan", "displayName": "ada",
            "email": "<university email>", "role": "student",
            "passwordHash": "scrypt$<salt hex>$<hash hex>" },
  "eventId": "6f1c...-uuid", "version": 42 }

{ "type": "user.deleted", "userId": 7, "eventId": "...-uuid", "version": 57 }
```

- `type`: one of `user.created`, `user.updated` or `user.deleted`.
- `user`: a full snapshot of the user after the change (*event-carried state transfer*). Consumers never need to call user-service to fill in details.
- `passwordHash`: included because `user-event-worker` rebuilds the user-service login table from these events. The other three consumers ignore it; the supplier and order copies keep only `id`, `displayName` and `role`.
- `version`: comes from one database sequence shared by all users, so it increases across the whole system. Consumers compare it per user: "apply only if newer than the version I stored for this user".
- `eventId`: a random UUID. Credit-service uses it as the idempotency key for the account allocation.

5.2 `supplier-events`

Publisher. `supplier-service`, through the `supplier_events` outbox. `POST`, `PUT` and `DELETE /suppliers`, renames of a type or a building (one `supplier.updated` per supplier that uses it), and the seed script all insert outbox rows in the same transaction as the change. The ordering key is the supplier ID, and the attributes are `type` and `supplierId`.

Subscription. `order-service-supplier-events`, push, ordered.

```
{ "eventId": "<outbox row id>", "version": 5, "type": "supplier.updated",
  "supplier": { "id": "<uuid>", "name": "NUS Co-op", "type": "Retail",
    "buildingName": "COM2", "floor": "1", "locationDescription": null,
    "latitude": "1.294100", "longitude": "103.774000", "isActive": true,
    "imageUrl": null,
    "operatingHours": [ { "day": 1, "openingHrs": "08:30", "closingHrs": "18:00" } ] } }
```

- `eventId` is the outbox row's primary key, so a republished row carries the same ID.
- `version` is taken from `supplier_event_sequence` by the outbox row's default value. It is unique across all suppliers, and order-service stores it as part of the key `(supplier_id, version)`.
- `latitude` and `longitude` are strings because PostgreSQL `numeric` columns are returned as strings, and order-service stores them back into numeric columns.
- `supplier.deleted` carries only `supplierId`, and order-service stores a *tombstone* version.

5.3 `order-events`

Publisher. `order-service`, through the `order_events` table, which is at the same time the order history shown on the details page and the outbox. Every state change appends a row and updates the cached `orders.current_state` in one transaction. The ordering key is the order ID; the attributes are `type` and `orderId`.

Subscriptions. `saga-orchestrator-order-events` is a push subscription with the filter `attributes.type = "order.created" OR ... "order.cancelled" OR ... "order.expired" OR ... "order.completed"`. Every running notification-service instance has its own pull subscription `notification-<uuid>`, and all instances share one pull subscription `notification-store`; both use the filter `hasPrefix(attributes.type, "order.")`.

```
{ "eventId": "<uuid>", "type": "order.created", "orderId": "<uuid>",
  "sequence": 1, "state": "PENDING", "actorId": 7, "commandId": null,
  "occurredAt": "2026-10-19T03:15:22.481Z",
  "order": { "id": "<uuid>", "requesterId": 7, "courierId": null,
    "supplierId": "<uuid>", "supplierVersion": 5, "title": "Two kopi",
    "description": "Less sugar", "deliveryLocation": "COM1 foyer",
    "creditReward": 12, "state": "PENDING", "acceptedAt": null,
    "expiresAt": null, "createdAt": "2026-10-19T03:15:22.470Z" } }
```

- `type` is one of `order.created`, `opened`, `rejected`, `aborted`, `accepted`, `cancelled`, `expired`, `picked_up`, `delivered`, `completed`, `settled` or `edited`, all prefixed with `order.`.
- `sequence` counts per order (1, 2, 3, ...). It is computed as "last + 1" under the order's row lock, and a unique constraint on `(order_id, sequence)` rejects any duplicate.
- `actorId` is the user who caused the change, or `null` for system changes (saga commands, expiry). Notification-service uses it so that nobody is notified about their own action.
- `commandId` is set when a saga command caused the change; it is unique in the table, so the same command cannot be applied twice.
- `order` is the full order snapshot after the change, which is why the saga can read `requesterId`, `courierId` and `creditReward` without calling order-service.

5.4 `credit-commands`

Publisher. `saga-orchestrator`, through its `outbox` table. The ordering key is the order ID, and the only attribute is `type`.

Subscription. `credit-service-commands`, push, created by credit's provisioning step *without*

message ordering.

```
{ "type": "credit.reserve",  "requestId": "<uuid>", "correlationId": "<uuid>",
  "orderId": "<uuid>", "requesterId": 7, "amount": 12 }
{ "type": "credit.release",  "requestId": "<uuid>", "correlationId": "<uuid>",
  "orderId": "<uuid>", "requesterId": 7, "amount": 12, "reason": "Order cancelled" }
{ "type": "credit.transfer", "requestId": "<uuid>", "correlationId": "<uuid>",
  "orderId": "<uuid>", "courierId": 9 }
```

- `requestId` is credit's idempotency key. The saga generates it once per step and stores it (`reserve_request_id`, `release_request_id`, `transfer_request_id`), so a republished command carries the same ID.
- `correlationId` is fixed per saga and is copied into every credit result.
- A transfer carries no amount. Credit uses the amount and requester it recorded when the reservation was made.

5.5 `credit-events`

Publisher. `credit-publisher`, a separate process that reads `credit_outbox` about once a second. The ordering key is the `orderId`, or the requester's user ID for account allocations, which have no order. The attributes are `type` and, when present, `orderId`.

Subscription. `saga-orchestrator-credit-events`, push, ordered.

```
{ "eventId": "<uuid>", "type": "credit.reserved",
  "occurredAt": "2026-10-19T03:15:23.104Z",
  "requestId": "<same as the command>", "correlationId": "<same as the command>",
  "orderId": "<uuid>",
  "data": { "requestId": "<uuid>", "status": "SUCCEEDED", "httpStatus": 200,
    "code": "OK", "orderId": "<uuid>", "reservationState": "HELD",
    "applied": true, "transactionIds": ["<uuid>"], "requesterId": 7, "amount": 12,
    "deltas": { "requesterTotal": 0, "requesterReserved": 12, "courierTotal": 0 } } }
```

- `type` is chosen from the outcome. A successful change produces `credit.reserved`, `credit.released`, `credit.transferred` or `credit.account_allocated`. A refused reservation produces `credit.reservation_rejected`; another refused command produces `credit.order_rejected`; a success that changed nothing produces `credit.order_succeeded`.
- `data.status` (`SUCCEEDED` or `REJECTED`) and `data.code`, such as `OK` or `INSUFFICIENT_CREDITS`, are what the saga actually reads, together with `requestId` and `orderId`.
- `deltas` records how the requester's total and reserved credits and the courier's total changed.

5.6 `order-commands`

Publisher. `saga-orchestrator`, through its outbox, with the order ID as ordering key and `type` as attribute. **Subscription.** `order-service-order-commands`, push, ordered.

```
{ "commandId": "<uuid>", "type": "OpenOrder", "orderId": "<uuid>" }
```

The four types map to fixed transitions in order-service: `OpenOrder` (PENDING→OPEN), `RejectOrder` (PENDING→REJECTED), `AbortOrder` (PENDING→ABORTED) and `SettleOrder` (COMPLETE→SETTLED).

> **Why it matters.** Because each payload is a full snapshot with its own identifiers, a consumer can be written, tested and restarted on its own: it needs only the message, never a callback to the publisher. The identifiers inside the payload (`version`, `eventId`, `commandId`, `requestId`) are what make duplicates harmless.

6 How a delivery is handled in code: push, pull and acknowledgements

6.1 What a push request looks like

With push delivery, Pub/Sub sends an HTTP `POST` whose JSON body wraps the message. The application data arrives base64-encoded in `message.data`:

```
{ "message": { "data": "eyJ0eXBlIjoidXNlci5jcmVhdGVkIiwi...",
               "attributes": { "type": "order.created", "orderId": "<uuid>" },
               "messageId": "...", "publishTime": "...", "orderingKey": "<uuid>" },
  "subscription": "projects/<project>/subscriptions/<name>" }
```

Every push handler in the repository reads only `message.data`, decodes the base64, and parses the JSON:

```
// order-service/src/events/pubsub.ts (same helper in the saga and supplier worker)
export function decodePushMessage(body: unknown): unknown {
  const message = typeof body === "object" && body !== null ? body.message : undefined;
  const data = typeof message === "object" && message !== null ? message.data :
    undefined;
  if (typeof data !== "string") return undefined;
  try { return JSON.parse(Buffer.from(data, "base64").toString("utf8")); }
  catch { return undefined; }
}
```

6.2 The HTTP status is the acknowledgement

For push subscriptions, the response status decides whether Pub/Sub retries. Order-service, the saga orchestrator and the supplier worker share one policy:

```
// order-service/src/app.ts (excerpt)
function pushHandler(name, handle) {
  return async (req, res) => {
    const message = decodePushMessage(req.body);
    try {
      if (message === undefined) throw new InvalidMessageError("Push request has no JSON
     message data");
      await handle(message);
    } catch (error) {
      if (!(error instanceof InvalidMessageError)) throw error;  // -> 500 -> retry
      console.error(`[order-service] Dropped invalid ${name} message: ...`);
    }
    res.status(204).end();                                          // ack
  };
}
```

Read it causally:

- If the message is *malformed*, it can never succeed, so the handler logs it and still answers 204. That acknowledges the message and stops it from blocking its ordering key forever.
- If the *database* fails, the error propagates to Express's error handler, which answers 500. Pub/Sub then redelivers later, by which time the database may be back.
- If the request fails the internal-authentication check, the answer is 401, which Pub/Sub also treats as a failure and retries.

The other two push consumers behave differently, and the difference matters:

- `user-event-worker` answers **400** for a malformed event, 500 for a database error, and 204 on success.
- `credit-service` answers 200 or 204 on success, **400** for an invalid event or command, 409 for an idempotency conflict, and 503 for transient conditions such as `ACCOUNT_PENDING`, `RESERVATION_NOT_READY` or a database error.

Pub/Sub treats a 400 exactly like a 500: the message is redelivered. Neither the code nor the deployment script configures a dead-letter topic, so a permanently malformed message on these two subscriptions is retried until the subscription's retention period ends. Credit's provisioning sets that to 7 days. For credit, retrying a 503 is intentional: a reservation for a user whose account has not been allocated yet waits, through Pub/Sub's redelivery, until the `user.created` event has been applied.

6.3 Pull delivery in notification-service

Notification-service creates two pull subscriptions when it starts and registers message listeners:

```
// notification-service/src/subscription.ts (excerpt)
const name = `${config.pubsub.subscriptionPrefix}-${randomUUID()}`;   // e.g.
    notification-1b9e...
const [live] = await topic.createSubscription(name, {
  filter: 'hasPrefix(attributes.type, "order.")', enableMessageOrdering: true,
  messageRetentionDuration: { seconds: 600 },          // live updates only
  expirationPolicy: { ttl: { seconds: 24 * 60 * 60 } }, // removed if the instance
    vanishes
});
live.on("message", (message) => { routeOrderEvent(parse(message)); message.ack(); });

store.on("message", async (message) => {               // shared "notification-store"
  try { await storeNotifications(notificationsForOrderEvent(parse(message))); message.
    ack(); }
  catch { message.nack(); }                            // redelivered later
});
```

The live subscription is acknowledged whether or not the user is online, because a user who connects later re-fetches stored notifications. The store subscription is acknowledged only after the insert succeeds. The insert is idempotent because the notification ID is a hash of `("order", eventId, userId)` and the table has a unique key on `(source, event_id, user_id)`. On shutdown (`SIGTERM`) the instance closes its sockets with code 1001 so that browsers reconnect elsewhere, and it deletes its own live subscription.

> **Why it matters.** For push, "what status do I return?" *is* the acknowledgement decision. Reading each handler with that question in mind tells you which failures heal themselves through retries and which messages could be retried for a long time.

7 The outbox pattern as implemented here

7.1 Writing: the change and the message row commit together

Here is supplier creation, the clearest example:

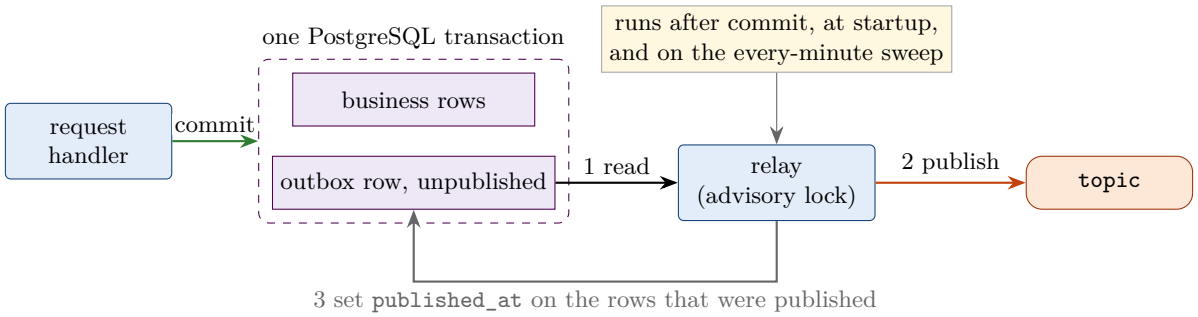


Figure 5: The outbox as used by supplier-, order- and saga-service. Credit-service has the same shape, but its relay is a separate process that loops every second and marks each row right after publishing it.

```
// supplier-service/src/index.ts, POST /suppliers (excerpt)
const newId = await db.transaction(async (tx) => {
  const row = await toSupplierRow(tx, input);
  const [created] = await tx.insert(suppliers).values({ ...row, isActive: row.isActive
    ?? true })
    .returning({ id: suppliers.id });
  if (hours.length > 0) await tx.insert(operatingHours).values(/* ... */);
  await recordSupplierChanges(tx, "supplier.created", eq(suppliers.id, created.id)); //
    outbox row
  return created.id;
});
requestOutboxPublish();   // start the relay after commit; do not wait for it
```

`recordSupplierChanges` reads the supplier's snapshot inside the same transaction `tx` and inserts it into `supplier_events`. If anything throws, PostgreSQL rolls back the supplier, its hours and the outbox row together. The same structure appears in order-service (`appendEvent` inserts into `order_events` inside the transaction that updates `orders`), in the saga (each step inserts into `outbox` inside the transaction that updates `sagas`), and in credit-service (`enqueueOutcome` inserts into `credit_outbox` inside the transaction that moves credits).

7.2 Relaying: at least once, in order per key

All three HTTP-service relays, `publishPendingSupplierEvents`, `publishPendingEvents` and `publishPending`, follow the same steps:

1. Open a transaction and try a PostgreSQL advisory lock (`pg_try_advisory_xact_lock`). If another relay holds it, return immediately. That is how only one relay runs per database at a time, which keeps each key's messages in sequence.
2. Select up to 100 unpublished rows in order (by `version` for suppliers, by creation time and sequence for orders, by creation time for the saga).
3. Publish them one by one with their ordering key. If a publish fails, add the key to a `blocked` set and skip that key's later rows in this run, so they cannot overtake the failed one.
4. Set `published_at` for the rows that were published, then commit.

The relay is started in three ways: right after the API's own commit (`requestOutboxPublish` or `requestPublish`, which runs again if more commits arrived during a run), once at process start, and by an every-minute sweep. The sweep runs in `supplier-ticker`, `order-ticker` and `saga-ticker` locally, and in Cloud Scheduler jobs in Google Cloud.

Credit-service uses a different relay. `credit-publisher` is a standalone process that loops forever: it takes a session-level advisory lock, publishes up to 100 rows in creation order, marks each one

immediately after its publish succeeds, and *stops the batch at the first failure*. Then it sleeps one second.

7.3 What this does and does not guarantee

- **Guaranteed by the code:** a business change and its outbox row are written in the same PostgreSQL transaction, so a committed change always has a message row and a rolled-back change never does.
- **Not guaranteed: exactly-once publication.** If the process crashes after Pub/Sub accepted a message but before `published_at` is committed, the next run publishes the same row again with the same `eventId`. The code comments state this, and the consumers deduplicate.
- **Not guaranteed: prompt publication.** If the after-commit relay fails, the message waits for the next sweep, which is up to a minute later.
- **Ordering** holds per ordering key, and only for subscriptions created with ordering enabled. The relays preserve per-key order; Pub/Sub preserves it on delivery.

User-service is the exception. It has no outbox. Its API publishes to Pub/Sub *instead of* writing the database; the worker writes the database later from the event. If the publish fails, the request fails and nothing changes anywhere. If it succeeds, the response is sent, but the user's own row appears only after `user-event-worker` applies the event. The repository README warns that immediate reads or logins can briefly lag. The administrator user page polls `GET /users/events/version`, the highest version applied so far, to refresh when the projection catches up.

Watch out. User-service checks e-mail and display-name uniqueness against its projected `users` table before it publishes. Because the projection lags, two conflicting registrations very close together could both pass that check. The worker's upsert is keyed by user ID, and the e-mail column has a unique index, so the second insert would fail and the worker would answer 500, which Pub/Sub retries. The code has no guard for this case. It is worth knowing about when you reason about user-events.

Why it matters. "Write the change and the intention to announce it atomically, then announce at least once" is what lets order-service promise that every committed state change eventually reaches the saga and the notification service, even if Pub/Sub was unavailable at the moment of the request.

8 Walkthrough A: a sign-up replicates the user to four services

This flow is fully implemented. It is the prerequisite for everything else: a user can create an order only after order-service knows them, and can reserve credits only after credit-service has allocated an account.

1. The sign-up page posts to the Nuxt route, which calls `user-service POST /register` at `http://user-service:3333`.
2. User-service validates the fields and checks uniqueness against its `users` table. It reserves the next user ID from the table's serial sequence and the next event version from `user_event_sequence`, and builds the user object, including an scrypt password hash, in memory. Nothing is inserted into `users`.
3. It publishes `user.created` with ordering key `"7"` and waits until Pub/Sub accepts it.
4. It returns 201 with a JWT. Nuxt stores it in the `user_access_token` cookie.
5. `user-event-worker` receives the push on `/pubsub/user-events`. In one transaction it upserts `user_event_states(user_id, version, deleted)`, but only if the new version is greater, and then inserts or updates the `users` row with the password hash. If the state upsert changed

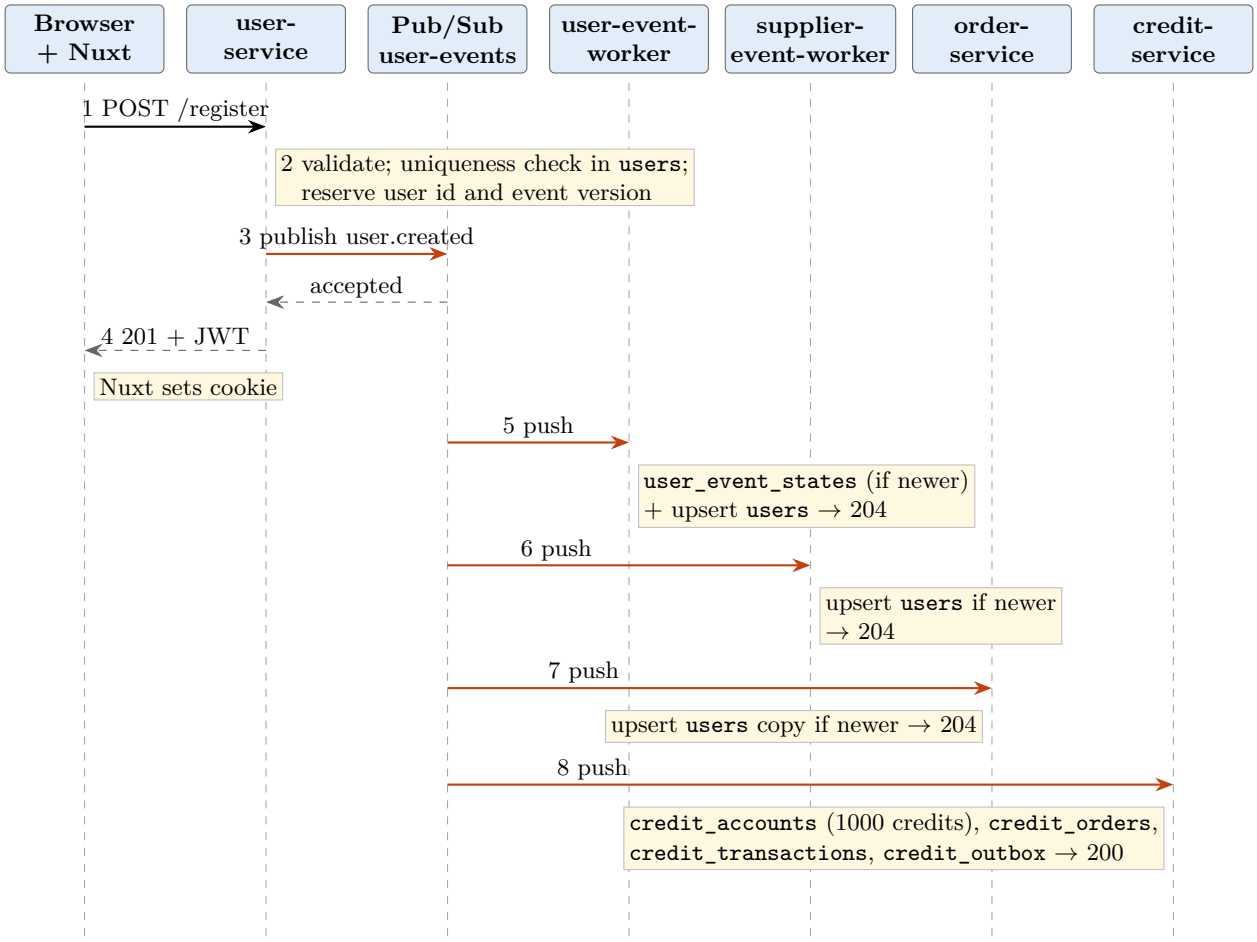


Figure 6: Registration. Steps 5–8 happen independently and in any order relative to each other; the browser already has its token after step 4.

nothing, the event is stale or a duplicate, and it commits without touching `users`. From this moment, `POST /login` can find the user.

6. `supplier-event-worker` upserts its `users` row (id, display name, role, version) with the condition "stored version < new version".
7. `order-service` does the same in its `users` table.
8. `credit-service` ignores `user.updated` entirely. For `user.created` it uses the `eventId` as request ID. Unless an account already exists, it creates one with `INITIAL_CREDITS` (default 1000) and records the grant as a credit order and a ledger transaction. It also writes a `credit.account_allocated` outbox row, which `credit-publisher` later publishes to `credit-events`. The saga ignores that event because it has no `orderId`, and no other consumer exists.

Deletion. `user.deleted` carries only `userId`. The user-event worker deletes the login row. The supplier and order copies *soft-delete*: they set `deleted_at` and keep the row, so existing orders can still reference the user. If a deletion arrives before the creation, it is stored as a tombstone row with a higher version, and the older creation is skipped when it arrives. Credit marks the account `deleted_at`.

> **Why it matters.** This one flow explains the 503 "still being set up" responses, the credit balance showing "being set up", and why compose makes `user-service` start only after the consumers that create subscriptions are healthy (Section 15.2). A subscription created after step 3 would never receive this user.

9 Walkthrough B: an order from creation to settlement

This is the main implemented business flow. It involves two topics of events, two topics of commands, four databases, and the notification service.

9.1 State machines first

Order-service owns the order's state. Users can trigger some transitions; saga commands trigger the others:

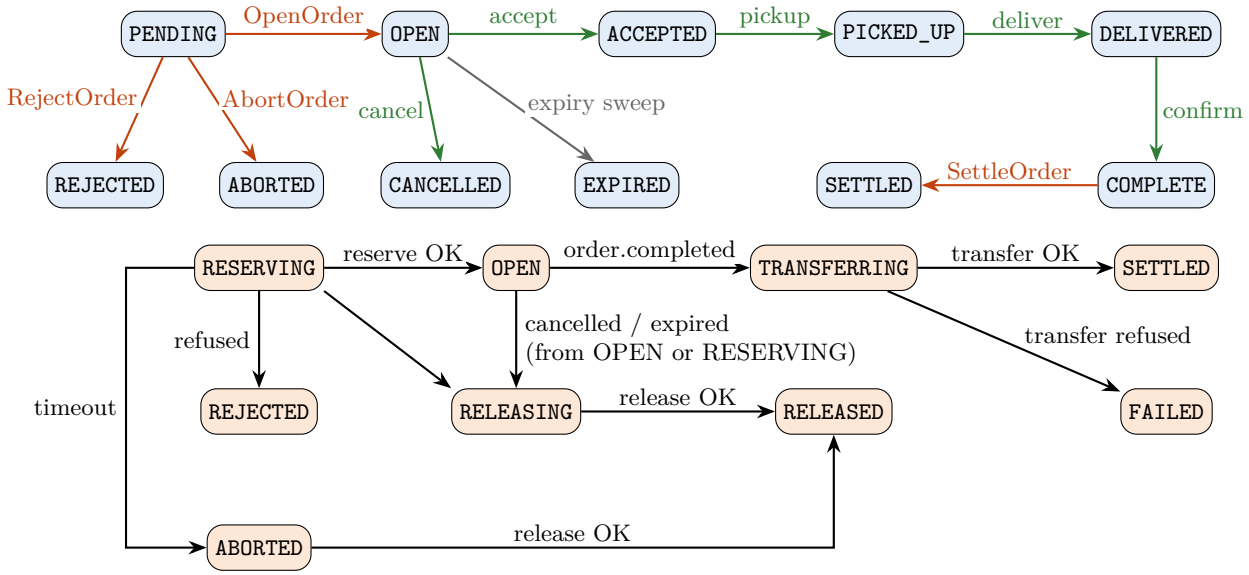


Figure 7: Top: order states in order-service (green = user action over HTTP, orange = saga command over Pub/Sub, grey = scheduled sweep). Bottom: saga states in saga-orchestrator. A refused release also leads to FAILED (not drawn).

9.2 Phase 1: creating and opening the order

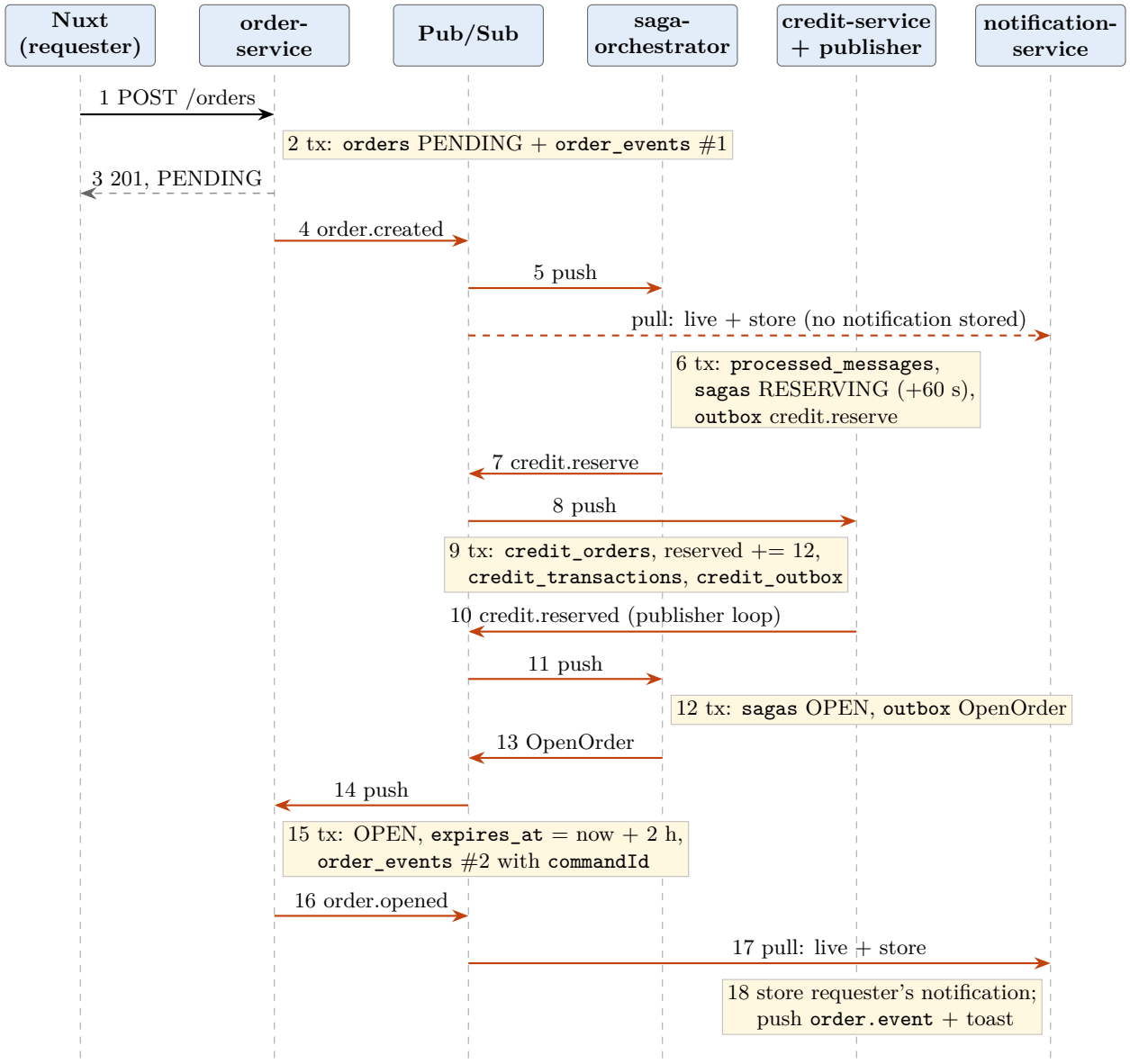


Figure 8: Opening an order. The saga sees `order.opened` too, but its subscription filter drops it.

1. The requester submits the form. Nuxt calls `POST /orders` with the bearer token. `requireUser` needs the requester in order-service's user copy, otherwise it answers 503.
2. Order-service looks up the *latest* row for the supplier in `supplier_versions`. It refuses if the supplier is unknown, deleted or inactive. Then, in one transaction, it inserts the order as `PENDING`, pointing at that supplier version, and appends `order_events` row 1 of type `order.created`.
3. The response is 201 with state `PENDING`. The user sees a pending order: the credit check has not happened yet.
4. The relay publishes the row to `order-events` (key = order ID, `type=order.created`).
5. The saga's filtered push subscription delivers it to `/internal/pubsub/order-events`. The notification instances also pull it; the live subscription pushes an `order.event` message to the requester's open sockets so pages refresh, but `order.created` creates no stored notification.
6. In one transaction the saga records the `eventId` in `processed_messages` (a repeat would stop here as "duplicate"). It inserts the saga row in state `RESERVING` with a fresh `reserve_`

request_id and a deadline 60 seconds ahead (RESERVE_TIMEOUT_SECONDS), and inserts a credit.reserve outbox row.

7. The saga relay publishes the command to credit-commands.
8. Credit-service receives it on /internal/pubsub/credit-commands.
9. In one transaction it takes advisory locks on the request, order and account. It returns the stored result if this requestId was seen before. Otherwise it checks that available credits (total minus reserved) cover the amount, moves 12 credits from available to reserved, records the credit order and a ledger transaction, and inserts a credit_outbox row. It answers 204.
10. Within about a second, credit-publisher publishes credit.reserved to credit-events.
11. The saga receives the result on /internal/pubsub/credit-events.
12. It matches requestId to the saga's reserve_request_id. Because the saga is still RESERVING and the status is SUCCEEDED, it moves the saga to OPEN and writes an OpenOrder outbox row.
13. The relay publishes OpenOrder to order-commands.
14. Order-service receives it on /internal/pubsub/order-commands.
15. Under a row lock it checks that no order_events row has this commandId and that the order is still PENDING. It then moves the order to OPEN, sets expires_at two hours ahead (ORDER_OPEN_TTL_SECONDS), and appends event 2 with the commandId.
16. The relay publishes order.opened.
17. Notification-service pulls it on both subscriptions.
18. The store subscription inserts a notification for the requester, because order.opened is a requester event and the actor is null. The live one sends both the raw order.event and a toast-ready notification to the requester's sockets. The order page refreshes and shows OPEN.

9.3 Phase 2: doing the errand and paying the courier

- **Accept, pickup, deliver, confirm** are ordinary HTTP calls to POST /orders/:orderId/<action>. Each one locks the order row, checks who may act (you cannot accept your own order; only the courier picks up and delivers; only the requester confirms), checks the current state, and appends an event. An accept after expires_at is refused even if the expiry sweep has not run yet.
- **The saga ignores accept, pickup and deliver**; its filter only passes order.completed. On order.completed it needs the saga in OPEN and a courierId in the snapshot. It then stores the courier, moves to TRANSFERRING and sends credit.transfer.
- **Credit** reads the reservation from its own records (amount and requester), refuses a transfer to the requester themselves, moves the reserved credits from the requester to the courier, and publishes credit.transferred.
- **The saga** matches the transfer's request ID, moves to SETTLED, and sends SettleOrder. Order-service moves COMPLETE→SETTLED and publishes order.settled, which notifies the courier.

Why it matters. Credits move only through the saga, never through an HTTP call from order-service. Order-service therefore never has to know credit balances. When credit is slow or unavailable, an order simply stays PENDING until the saga decides, as the next section shows.

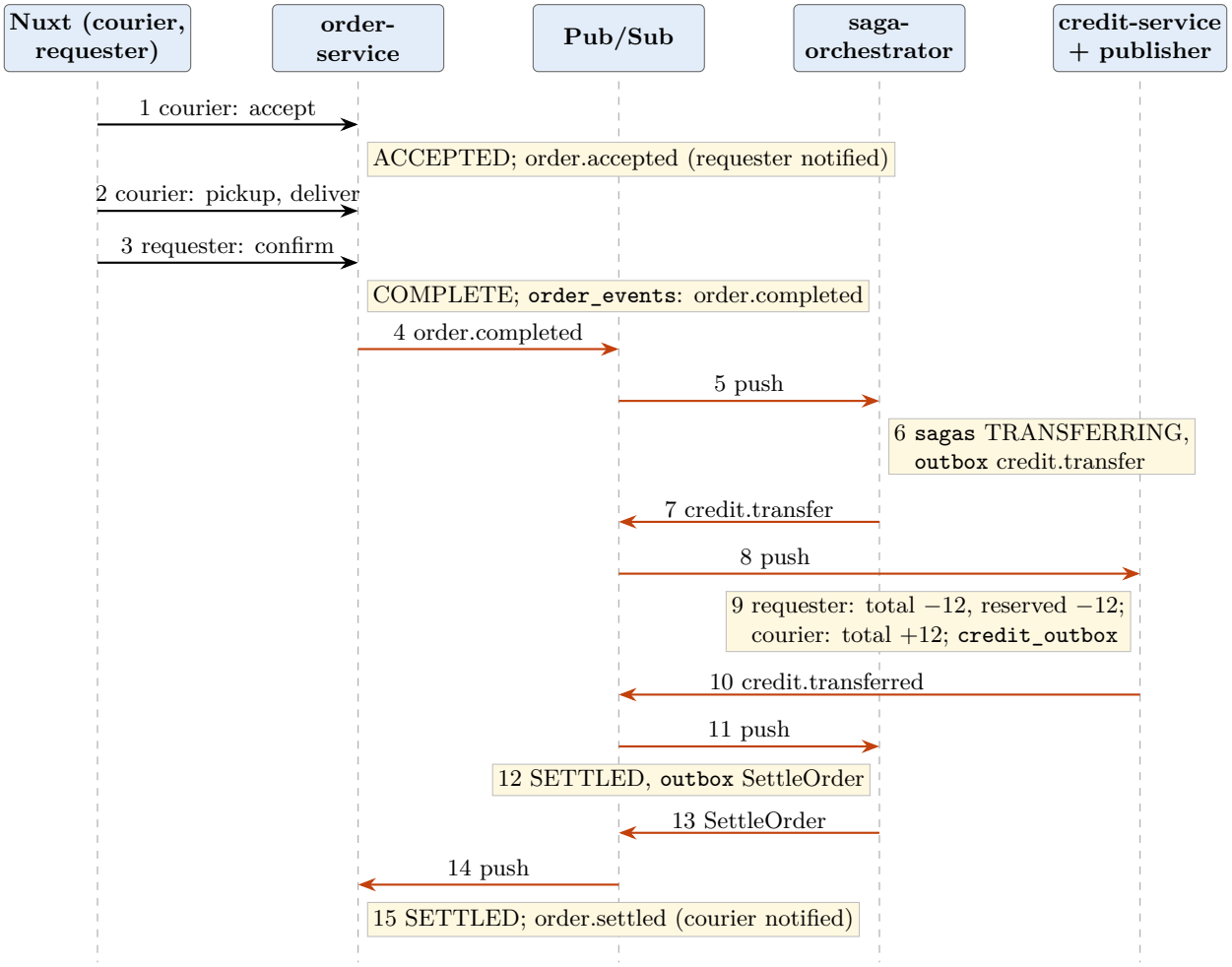


Figure 9: Completing an order. The final `order.settled` event notifies the courier (not drawn).

10 Walkthrough C: refusals, timeouts, compensation and duplicates

10.1 Not enough credits: the order is rejected

If the requester's available credits are below the reward, credit's reserve decision returns code `INSUFFICIENT_CREDITS` with status `REJECTED` and changes no balance. The publisher emits `credit.reservation_rejected`. The saga, still in `RESERVING`, stores the failure code, moves to `REJECTED` and sends `RejectOrder`. Order-service moves `PENDING→REJECTED` and publishes `order.rejected`, which notifies the requester. Nothing needs to be compensated, because nothing was reserved.

10.2 Credit does not answer in time: abort and release

Suppose credit-service is down. The `credit.reserve` command waits in its subscription, and the saga row stays `RESERVING`. Every minute `saga-ticker` (Cloud Scheduler in Google Cloud) calls `POST /internal/sagas/abort-overdue`:

```
// saga-orchestrator/src/saga.ts, abortOverdueReservations (excerpt)
const overdue = await tx.select().from(sagas)
  .where(and(eq(sagas.state, "RESERVING"), lte(sagas.reserveDeadline, now)))
  .orderBy(asc(sagas.reserveDeadline)).limit(BATCH)
  .for("update", { skipLocked: true });
for (const saga of overdue) {
  await release(tx, saga, "ABORTED", "Credit reservation timed out"); // credit.release
    command
  await orderCommand(tx, saga.orderId, "AbortOrder");                  // order-commands
}
```

This is the compensating step. The saga does not know whether the reservation happened, so it sends a release either way, and order-service moves the order to `ABORTED`. When credit comes back, Pub/Sub redelivers the queued commands, and both cases are safe:

- If the *release* is processed first, credit has no reservation for the order, so it records the order as `CANCELLED` and reports success; the saga moves `ABORTED→RELEASED`. The late reserve is then refused with `RESERVATION_CANCELLED`, and the saga ignores that result because it is no longer `RESERVING`.
- If the *reserve* is processed first, credits are held. The release then frees them (`HELD→RELEASED`), and the saga moves `ABORTED→RELEASED`.

Because `credit-service-commands` is not an ordered subscription, either order can happen. Credit's per-order advisory lock and its reservation-state checks make both orders safe.

10.3 Cancelled or expired orders: release

A requester can cancel only an `OPEN` order. Order-service's expiry sweep (`POST /internal/orders/expire-overdue`, every minute) moves `OPEN` orders past `expires_at` to `EXPIRED`, using `SKIP LOCKED` so it does not fight a concurrent accept. Either event reaches the saga, which moves `OPEN` (or `RESERVING`) to `RELEASING` and sends `credit.release`. A successful release moves the saga to `RELEASED`; the order itself stays `CANCELLED` or `EXPIRED`, and no further order command is sent.

10.4 Waiting for prerequisites through retries

Two credit conditions are answered with 503 on purpose, so that Pub/Sub keeps retrying (between 1 and 30 seconds apart, per credit's subscription settings):

- `ACCOUNT_PENDING`: the account does not exist yet because `user.created` has not been applied.

- `RESERVATION_NOT_READY`: a transfer arrived before credit had recorded any reservation for the order.

10.5 Duplicates, layer by layer

- **Replicated users** (user, supplier, order copies): "apply only if the stored version is lower". A repeat changes nothing.
- **Supplier versions** (order-service): primary key `(supplier_id, version)` with "on conflict do nothing".
- **Saga**: `processed_messages` keyed by `eventId`. Credit results are also matched against stored request IDs and the current saga state.
- **Order commands**: unique `order_events.command_id`, checked under the order's row lock.
- **Credit commands**: `credit_orders.request_id`. A repeat with the same payload returns the stored result and publishes nothing new; the same `requestId` with a different payload is a 409 `IDEMPOTENCY_CONFLICT`.
- **Account allocation**: the `user.created` `eventId` is the request ID, and a partial unique index allows one successful allocation per user.
- **Notifications**: ID derived from (source, event, user) plus a unique key.

10.6 What is not handled automatically

The saga state `FAILED` is reached if credit refuses a release or a transfer, for example `USER_INACTIVE` for a deleted courier or `BALANCE_LIMIT_EXCEEDED`. The code logs an error, and the schema comment calls it "needs attention". No code path retries, alerts or compensates further. The order stays `COMPLETE` (for a failed transfer) or in its cancelled/expired state.

> **Why it matters.** The saga never assumes a message arrived, or arrived once, or arrived in a convenient order. Each decision is made from its own stored state plus the identifiers in the message. That is what makes the timeout-and-release design safe.

11 Walkthrough D: a supplier edit reaches order-service

1. An administrator saves the edit dialog. Nuxt calls supplier-service `PUT /suppliers/:id`. `requireAuthentication` verifies the JWT and reads the role from the supplier database's local `users` copy; `requireAdministrator` demands `admin`.
2. In one transaction the supplier row is updated, its operating hours are replaced if they were sent, and a `supplier.updated` outbox row with the full snapshot is inserted.
3. After the commit the API starts its relay, which publishes to `supplier-events` with key = supplier ID. If that fails, the row waits for the worker's `POST /outbox/publish`, called every minute by `supplier-ticker` locally or by the `supplier-outbox-sweep` Cloud Scheduler job in Cloud.
4. Order-service's `/internal/pubsub/supplier-events` handler appends a new row to `supplier_versions`. It never updates old rows. Image URLs are deliberately not stored.
5. The next `POST /orders` for this supplier uses the new latest version. Existing orders keep their `supplier_version` foreign key, so they still describe the supplier as it was when they were placed.

Renames of a type or building lock every supplier row that uses it, then emit one `supplier.updated` per supplier. **Deleting** a type or building emits nothing; it is allowed only when no supplier uses the row. **Seeding** (`npm run db:seed`) inserts outbox rows but publishes nothing itself, so locally the

seeded suppliers reach order-service on the next `supplier-ticker` sweep, within about a minute. Until then, creating an order for a seeded supplier fails with "Supplier not found".

12 Local emulator vs Google Cloud transport

12.1 Locally: services set up their own topics and subscriptions

The emulator container starts empty and keeps everything in memory; no volume is mounted. Each consumer therefore creates what it needs at startup, but only when `PUBSUB_EMULATOR_HOST` is set, which the client library also uses to connect to `pubsub-emulator:8085` instead of Google:

- `user-event-worker`: topic `user-events` and push subscription `user-events-worker` to `http://user-event-worker:3334/pubsub/user-events`.
- `supplier-event-worker`: topics `user-events` and `supplier-events`, and push subscription `supplier-service-user-events`. If that subscription already exists, it re-points its endpoint.
- `order-service`: topics `order-events`, `user-events`, `supplier-events` and `order-commands`, and three push subscriptions to `http://order-service:3000/internal/pubsub/...?token=<secret>`.
- `saga-orchestrator`: its four topics, and the two push subscriptions, including the filter on the order-events one.
- `credit-pubsub-provision`: credit's three topics and two push subscriptions, with the token in the query string. It retries up to 30 times, one second apart.
- `notification-service`: topic `order-events`, its own `notification-<uuid>` pull subscription and the shared `notification-store`.

The emulator cannot attach Google OIDC tokens to push requests. So the push URLs carry a local-only shared secret as `?token=...`, and the tickers send it in the `x-internal-secret` header. Compose supplies defaults such as `local-only-order-internal-secret-0000`. Credit-service accepts the shared secret only while `PUBSUB_EMULATOR_HOST` is set, and accepts the `?token=` form only on its push routes. `user-event-worker` and `supplier-event-worker` have no authentication check in their code at all; they depend on not being reachable, which holds locally because they have no host port, and in Cloud because of Cloud Run IAM.

12.2 In Google Cloud: `deploy.sh` creates the transport

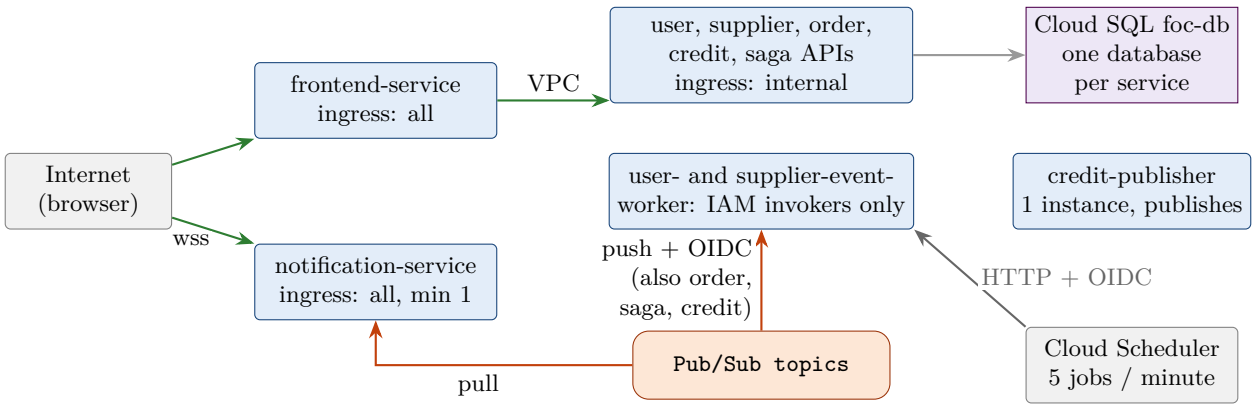


Figure 10: Cloud topology as configured in `deploy.sh`. Pub/Sub push and Scheduler also call the internal-ingress backends (arrows simplified). The supplier-service and its worker reach Cloud SQL over a private IP; the others use the Cloud SQL connector.

- **Topics** are created by `ensure_pubsub` (`user-events`, `supplier-events`) and by `ensure_service_resources` (`order-events`, `order-commands`, `supplier-events`, `credit-commands`, `credit-events`).
- **Push subscriptions** are created by `deploy.sh` with an OIDC token from the service account `user-event-push@<project>` and an audience equal to the service URL. The order and saga subscriptions are also created with an ack deadline of 30 seconds and a retry delay of 1–60 seconds. The exceptions are credit's two subscriptions, which the `credit-provision` Cloud Run job creates from the credit image with the same push identity. Message ordering is enabled on the order, saga, user-worker and supplier-worker subscriptions; the saga's filter is applied at creation and cannot be changed later.
- **Verifying push callers.** Order-service and the saga verify the Google ID token's signature and audience (`INTERNAL_AUDIENCE`) and require its e-mail to be in `INTERNAL_INVOKER_EMAILS`, which holds the push and scheduler accounts. Credit accepts only the push account on its push routes. The two event workers are deployed with `--no-allow-unauthenticated`, and the push account is granted `roles/run.invoker`, so Cloud Run itself rejects other callers.
- **Scheduler jobs**: `supplier-outbox-sweep`, `order-expire-overdue`, `order-outbox-publish`, `saga-abort-overdue` and `saga-outbox-publish`, every minute, each with an OIDC token.
- **Notification-service** pulls, so it needs CPU between requests: `--min-instances 1 --no-cpu-throttling --timeout 3600`, the timeout also limiting WebSocket lifetime. It creates and deletes subscriptions itself, so the default compute service account gets `roles/pubsub.editor`.
- **credit-publisher** loops in the background, so it runs as exactly one instance with CPU always on. It answers a trivial HTTP health response on `PORT`, which Cloud Run requires.

12.3 Configured versus actually exercised

Be precise about what has evidence behind it. For this guide nothing was run: the machine it was prepared on has no Docker, and no cloud calls were made. The repository's `AI_USAGE_LOG.md` records the following earlier checks:

- **Reported as run on the local emulator:** supplier create/edit/delete publishing versions that order-service stored; two notification instances both delivering an order event; and the full saga with the real user, order and credit services (opening, rejection for insufficient credits, cancellation release, completion transfer and settlement, and abort after a timeout with credit stopped).
- **Configured but not recorded as run:** the Cloud Pub/Sub transport. The entries for the order/notification/credit/saga deployment and for supplier events list only `bash -n deploy.sh` and config parsing, and say explicitly that "Cloud deployment was not run". An earlier Cloud deployment of frontend, user and supplier services is recorded as working, but it predates the Pub/Sub changes.

13 Containers from first principles

- An **image** is a frozen filesystem plus a default command. A **Dockerfile** builds it step by step; each `FROM` starts a *stage*, and a later stage can copy files out of an earlier one. Only the final stage, or the stage named by `target`, becomes the image.
- The **build context** is the folder sent to the builder. `COPY` can only see files in it, minus those listed in `.dockerignore`. Here every context is the service's own folder, so no image can accidentally include another service's code.
- A **container** is a running instance of an image. Compose's `command:` replaces the image's `CMD`;

that is how one image runs both an API and a worker.

- **`EXPOSE`** in a Dockerfile only documents a port. It opens nothing. **Publishing** (`ports: "3002:3000"` in Compose) is what makes host port 3002 forward to container port 3000. Containers on the same Compose network reach each other on the *container* port regardless of publishing.
- A **named volume** (`order-db-data:`) is storage managed by Docker that survives container re-creation. A **bind mount** (`./x:/y`) maps a host folder into the container.

> **Why it matters.** Two facts prevent most confusion here. First, the supplier worker listens on 3004 even though its image says `EXPOSE 3000`; `EXPOSE` is only documentation. Second, the host's port 3004 is credit-service, not the supplier worker. Host ports and container ports are separate namespaces.

14 Each Dockerfile, explained

14.1 The three-stage pattern: supplier, order, notification, saga

These four services use the same layout, shown here for order-service:

```
FROM node:22-bookworm-slim AS tooling
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci                                  # all dependencies, incl. drizzle-kit, tsx,
    tsc
COPY tsconfig.json tsconfig.build.json ./
COPY drizzle.config.ts ./
COPY src ./src
COPY drizzle ./drizzle                      # SQL migrations
CMD ["npm", "run", "db:migrate"]            # supplier-service: db:check

FROM tooling AS build
RUN npm run build && npm prune --omit=dev   # tsc -> dist/, then drop dev deps

FROM node:22-bookworm-slim AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=node:node /app/package.json /app/package-lock.json ./
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

- **Why copy `package*.json` before `src`?** Docker caches each step. If only source files change, the slow `npm ci` layer is reused.
- **`tooling`** keeps TypeScript sources, dev dependencies and migrations. Compose's `*-db-migrate` containers use `target: tooling` to run `drizzle-kit migrate`, and `compose.dev.yaml` uses it to run `tsx watch`.
- **`runtime`** contains only compiled JavaScript and production dependencies, and runs as the unprivileged `node` user. It is the default when no target is named, so `build: ./order-service` produces it.
- **Workers reuse the runtime image**: `supplier-event-worker` runs it with `command: ["node", "dist/worker.js"]`.

14.2 credit-service: two stages on Alpine

```
FROM node:22-alpine AS tooling
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.json drizzle.config.ts ./
COPY src ./src
COPY migrations ./migrations
RUN npm run build                       # dist/ built here
FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY --from=tooling /app/package*.json ./
RUN npm ci --omit=dev                   # reinstall production deps only
COPY --from=tooling /app/dist ./dist
COPY --from=tooling /app/migrations ./migrations
EXPOSE 3000
CMD ["npm","start"]
```

The differences from the pattern above have consequences. The base image is Alpine, not Debian slim. Production dependencies are reinstalled rather than pruned. There is no `USER` line, so the container runs as root. Migrations run with the *runtime* image: `npm run db:migrate` runs `node dist/db/migrate.js`, which uses Drizzle ORM's migrator (no dev dependency needed) under an advisory lock. The same single image therefore serves as API (`npm start`), publisher (`npm run publisher`), provisioner (`npm run pubsub:provision`) and migrator, both in Compose and in Cloud. Its `.dockerignore` also excludes `test` and `docs`.

14.3 user-service: one stage, TypeScript at runtime

```
FROM node:22-bookworm-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY drizzle.config.ts ./
COPY src ./src
ENV PORT=3333
EXPOSE 3333
CMD ["npm", "start"]                      # = tsx src/index.ts
```

There is no build step. The image keeps all dependencies, including dev ones, and runs TypeScript directly with `tsx`. It runs as root, and the folder has no `.dockerignore`, so the whole `user-service/` folder is the build context. The same image runs three ways: API (`npm start`), worker (`npm run worker` = `tsx src/worker.ts`), and migration. The migration script is `drizzle-kit push`, which compares `src/db/schema.ts` with the live database and applies the difference; there are no migration files in the folder.

14.4 frontend-service: build, then ship only the output

```
FROM node:22-bookworm-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build                          # nuxt build -> .output/
FROM node:22-bookworm-slim AS runtime
ENV NODE_ENV=production NITRO_HOST=0.0.0.0 NITRO_PORT=3000
WORKDIR /app
COPY --from=build --chown=node:node /app/.output ./.output
USER node
EXPOSE 3000
CMD ["node", ".output/server/index.mjs"]
```

Nuxt's `.output` folder is self-contained, so the runtime stage does not even copy `node_modules`. Backend URLs are not baked in at build time. They are read when the server starts from `NUXT_*` environment variables that override the defaults in `nuxt.config.ts`, which is why the same image works in Compose and Cloud. The `build` stage doubles as the development target.

14.5 Images used as-is

`postgres:17` for all six databases, `gcr.io/google.com/cloudsdktool/google-cloud-cli:emulators` for the Pub/Sub emulator (started with `gcloud beta emulators pubsub start --project=foc-local --host-port=0.0.0.0:8085`), and `curlimages/curl:8.11.1` for the three tickers, whose entrypoint is a shell loop of `sleep 60` and `curl -X POST`.

15 Compose: the running system

15.1 One network, names as addresses

`compose.yaml` declares no networks, so Compose puts every container on one default project network and registers each *service name* in its internal DNS. That is why configuration can say `postgres://order:order@order-db:5432/orders`, `PUBSUB_EMULATOR_HOST=pubsub-emulator:8085` or `NUXT_USER_SERVICE_BASE_URL=http://user-service:3333`. Container-to-container traffic always uses the container port: 5432 for every database, even though the host sees 5433–5439.

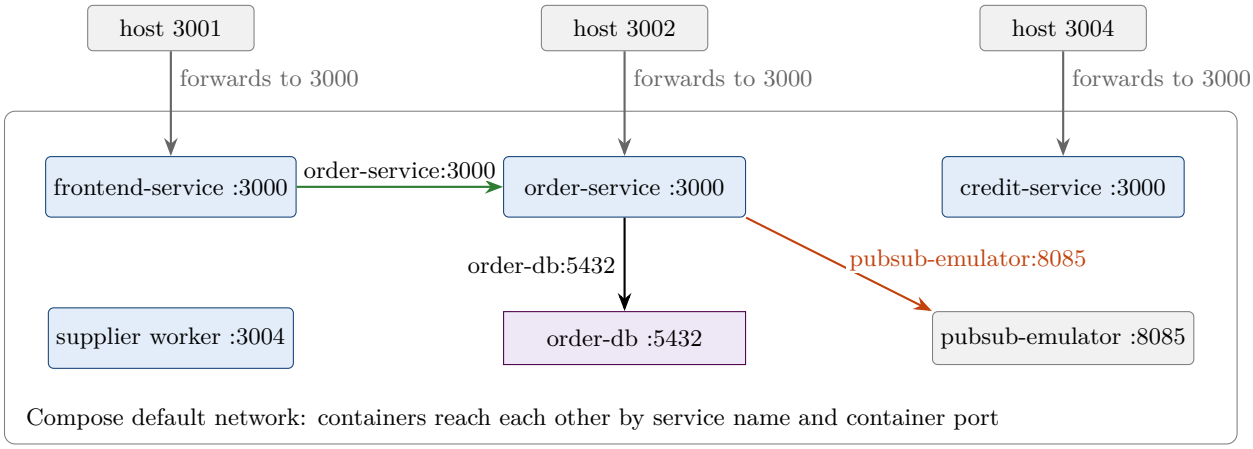


Figure 11: Inside the network, services address each other by name and container port. Published ports (top) are a separate door from the host. Only a selection is drawn: notification-service is also published (3003), as are the databases on 127.0.0.1 (5433–5439); the saga is not.

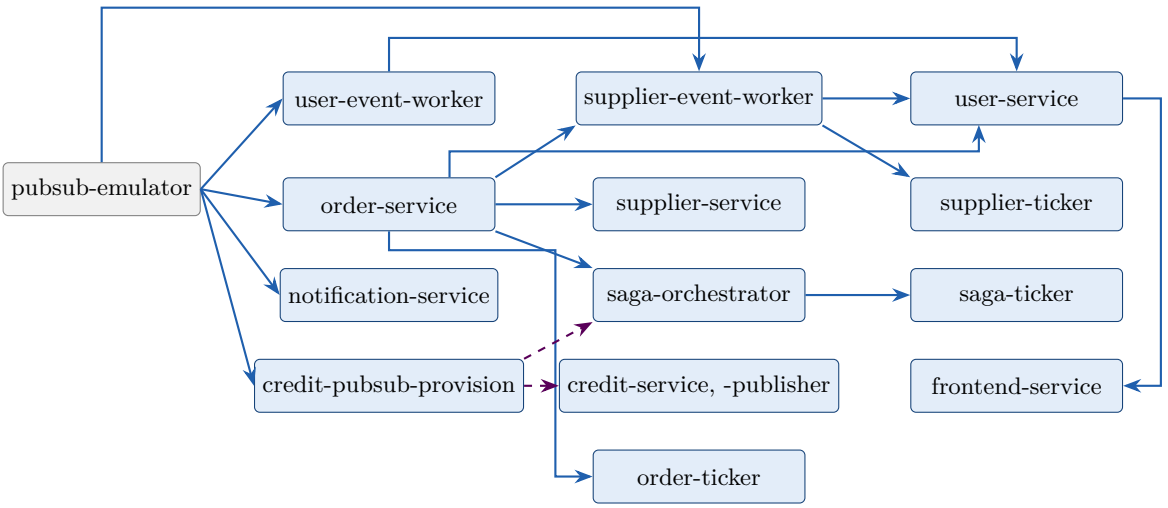
15.2 Startup order: `depends_on` is not the same as "ready"

`depends_on` only controls *when Compose starts* a container. By itself it waits only for the dependency to have started. This file always adds a condition:

- `service_healthy` waits until the dependency's `healthcheck` passes.
- `service_completed_successfully` waits until a one-shot container exits with code 0 (all `*-db-migrate` jobs and `credit-pubsub-provision`).

A healthcheck is a command Compose runs inside the container. The checks are:

- **PostgreSQL**: `pg_isready -U <user> -d <db>` every 5 s.
- **Emulator**: `curl -fsS http://127.0.0.1:8085/v1/projects/foc-local/topics`.
- **user, user-worker, supplier-worker, order, notification, saga**: a one-line `node -e "fetch('http://127.0.0.1:<port>/health')..."`. These `/health` routes return a fixed `ok`, so they prove the server is listening. The services only call `listen()` after their startup work is done: a database round trip for user-service and its worker, order, notification and saga; emulator topics and subscriptions for the user worker, supplier worker, order, notification and saga. So "healthy" also means "my subscriptions exist". The supplier worker does not query the database before listening.
- **credit-service**: `/health/ready`, which runs `SELECT 1`, so it also tests the database each time.
- **credit-publisher**: `node dist/events/publisher-health.js`, which queries the database and fetches the `credit-events` topic's metadata.
- `supplier-service`, `frontend-service`, the tickers and the one-shot jobs have *no* healthcheck, so nothing can wait for them to be healthy.



solid: waits for `service_healthy` **dashed**: waits for `service_completed_successfully`
Not drawn: each service also waits for its own `*-db-migrate` job (completed), which waits for its database (healthy).

Figure 12: Cross-service startup dependencies in `compose.yaml`.

The arrows are not arbitrary. Each one encodes "the subscription must exist before the first message is published":

- `user-service` waits for `user-event-worker`, `order-service` and `supplier-event-worker`, because those three create their `user-events` subscriptions at startup. Comments in the file say exactly this.
- `supplier-service` and `supplier-event-worker` wait for `order-service`, which creates

order-service-supplier-events.

- saga-orchestrator waits for order-service and credit-pubsub-provision, because its commands need subscribers on both sides.

> **Watch out.** user-service does *not* wait for credit-pubsub-provision, which creates credit-service-user-events. In practice provisioning finishes in seconds, long before anyone registers through the browser, but Compose does not enforce it. Also, supplier-service's volume entry mounts ./supplier-service/db/init into the database's init folder, and that folder does not exist in the repository. Docker creates an empty host directory, so no init scripts run.

15.3 Volumes and persistence

Each database stores /var/lib/postgresql/data in a named volume (supplier-db-data, user-db-data, order-db-data, notification-db-data, credit-db-data, saga-db-data). Data survives docker compose down and is lost only with down -v. The emulator has no volume, so topics, subscriptions and undelivered messages live only as long as its container. Bind mounts appear in two places: the empty supplier init folder described above, and the supplier-db-tools container (profile tools), which mounts src, drizzle and drizzle.config.ts from the host. That lets drizzle-kit generate write new migration files back into the repository.

15.4 Environment and secrets

Compose reads a root .env, created from .env.example. JWT_SECRET is mandatory: ${JWT_SECRET:?Set JWT_SECRET in .env} aborts startup if it is missing, and every service that verifies tokens checks that it is at least 32 bytes. Everything else has a default in the file: database URLs with fixed local passwords, INITIAL_CREDITS (1000), RESERVE_TIMEOUT_SECONDS (60), and the local-only internal secrets. Credit's variables are written once as a YAML anchor (&credit-environment) and reused by credit-service, credit-publisher and credit-pubsub-provision. That shared block sets PORT=3000, so the local publisher also starts its small health server, on port 3000 inside its own container.

16 compose.dev.yaml: hot reloading

Run as docker compose -f compose.yaml -f compose.dev.yaml up --watch. The second file *overrides* matching keys in the first:

- The supplier API and worker, order, notification, credit API and publisher, and saga switch to build.target: tooling, where sources and tsx exist, and run npx tsx watch src/<entry>.ts.
- user-service and user-event-worker keep their single-stage image and only change the command to tsx watch.
- frontend-service uses the build stage and runs nuxt dev --host 0.0.0.0 --port 3000 with NODE_ENV=development.
- Each one has a develop.watch block: action: sync copies edited files from ./<service>/src into /app/src inside the running container, where tsx watch or Nuxt reloads them, and action: rebuild rebuilds the image when package.json changes. This is file syncing, not a bind mount.
- Migrations, databases, the emulator, tickers and credit-pubsub-provision are not overridden.

17 Databases: ownership, persistence and migrations

user-db (`users`)
Tables `users`, `user_bootstrap_reservations`, `user_event_states`, and the sequence `user_event_sequence`. Written only by `user-event-worker`, apart from the bootstrap reservation. Migration: `drizzle-kit push` from the schema, with no migration files.

supplier-db (`suppliers`)
`types`, `buildings`, `suppliers`, `operating_hours`, the user copy `users`, and the outbox `supplier_events` with `supplier_event_sequence`. Migrations: three SQL files in `drizzle/` (0000 schema, 0001 local users, 0002 supplier events), applied with `drizzle-kit migrate`. Seeding is a separate, repeatable script.

order-db (`orders`)
`users` (copy), `supplier_versions` (append-only copy), `orders`, and `order_events` (history plus outbox). One migration file, applied with `drizzle-kit migrate`.

saga-db (`sagas`)
`sagas`, `processed_messages` and `outbox`. One migration file.

credit-db (`credits`)
`credit_accounts`, `credit_orders`, which holds each command's request, result and idempotency key, `credit_transactions` (ledger), and `credit_outbox`. One migration file in `migrations/`, applied by Drizzle ORM's migrator from the runtime image.

notification-db (`notifications`)
`notifications`. One migration file.

Locally every database is a separate `postgres:17` container with its own named volume, and each `*-db-migrate` container runs once before its service starts. In Cloud, `deploy.sh` creates one Cloud SQL instance `foc-db` with a separate database and database user per service. Each connection URL is stored in Secret Manager (for example `order-database-url`), and migrations run as Cloud Run jobs (`order-migrate`, `credit-migrate`, ...) before the services are deployed.

18 Cloud Build and Cloud Run packaging

`cloudbuild.yaml` builds eleven images in parallel (`waitFor: ["-"]`), each from its service folder as build context. They are tagged with the build ID and `latest` and pushed to Artifact Registry at `asia-southeast1-docker.pkg.dev/<project>/foc/`. Seven are runtime images: the frontend and the six backends (user, supplier, order, notification, credit, saga). Four are `-tooling` images (`--target=tooling`) for supplier, order, notification and saga, which carry `drizzle-kit` for the migration and seed jobs. User and credit have no tooling image because their own images can already migrate.

`deploy.sh` maps those images to Cloud Run like this:

- **One image, several Cloud Run resources.** `user-service` and `user-event-worker` (`--command npm --args run,worker`); `supplier-service` and `supplier-event-worker` (`node dist/worker.js`); and `credit-service`, `credit-publisher`, the `credit-migrate` job and the `credit-provision` job.
- **Ports.** `--port` tells Cloud Run which container port to send traffic to: 3333 for user, 3334 for the user worker, 3004 for the supplier worker, and 3000 for the rest. Cloud Run sets `PORT` to the same value.
- **Ingress.** The frontend and notification-service accept traffic from anywhere. The other APIs use `--ingress internal` and are reached by the frontend over Direct VPC egress, with Private Google Access enabled on the subnet. The two event workers and credit-publisher are

not publicly invokable.

- **Configuration.** Secrets (`DATABASE_URL`, `JWT_SECRET`, `NOTIFICATION_TICKET_SECRET`, optional `BOOTSTRAP_SECRET`) come from Secret Manager. Plain settings (`PUBSUB_PROJECT_ID`, `INTERNAL_AUDIENCE`, `INTERNAL_INVOKER_EMAILS`, ...) come from `--set-env-vars`. No `PUBSUB_EMULATOR_HOST` is set, so the services talk to real Pub/Sub, skip emulator self-provisioning, and do not accept the shared secret.

19 Gaps, missing links and things to keep in mind

These points are visible in the code at the pinned commit. They are described, not judged; whether any of them needs changing is a design decision for the team.

- **`credit-events` has one real consumer.** Only the saga subscribes, and it ignores `credit.account_allocated`. The notifications table allows a `credit` source, but no code produces credit notifications.
- **Two credit endpoints have no caller.** `GET /internal/v1/credit-orders/:requestId` and `GET /internal/v1/credit-reservations/:orderId` exist, but nothing in the repository calls them.
- **`FAILED` sagas stop.** A refused release or transfer leaves the saga in `FAILED` with only a log line.
- **No dead-letter topics.** `user-event-worker` and credit answer 400 to invalid messages, which Pub/Sub retries until retention ends. Order, saga and the supplier worker log and acknowledge them instead.
- **Credit's subscriptions are unordered**, although publishers set ordering keys. Credit relies on locks and reservation state instead of delivery order.
- **User-service has no outbox** and checks uniqueness against a lagging projection (Section 7).
- **Compose ordering gap**: user-service does not wait for credit's provisioning, and the supplier DB init mount points to a missing folder (Section 15.2).
- **Emulator state is ephemeral.** If only `pubsub-emulator` is restarted, subscriptions vanish until the services that create them restart too.
- **Cloud Pub/Sub has no recorded run** (Section 12).
- **Root README drift.** The repository-structure section lists only four services; the notification service and saga orchestrator, and the order and credit Pub/Sub flows, are described only in the service READMEs and Compose files.

Every frontend proxy route has a matching backend route: users, auth and bootstrap; suppliers, types and buildings; orders and actions; credit balance; and notification tickets and lists. No frontend call targets an unimplemented endpoint.

20 Notes for the upcoming event bus tap

The tap itself is a separate, later task. These are observations from the current code that its design will have to account for:

- **Fanout makes a tap non-invasive.** A new subscription per topic receives every message without taking any away from existing consumers. Joining an existing subscription would instead *compete* with its consumer and steal messages. The six topics are `user-events`, `supplier-events`, `order-events`, `order-commands`, `credit-commands` and `credit-events`.
- **A tap only sees messages published after its subscription exists**, and locally only while the emulator keeps running.

- **Reachability.** The emulator (8085) is not published to the host; a local tap would run on the Compose network or need a published port. In Cloud, a tap needs subscriber or editor permissions for its own subscriptions; notification-service's pattern of creating and deleting a per-instance subscription with an expiry policy is a working precedent.
- **Useful message metadata**: `type` attributes exist on all topics except `user-events`; ordering keys are user, supplier or order IDs; and `correlationId` and `requestId` link credit commands to results.
- **Sensitive data**: `user-events` payloads contain `passwordHash` and e-mail addresses, which a tap should not display or store as they are.
- **Pull vs push**: pull keeps a tap free of public endpoints and OIDC verification, as notification-service shows; push would need its own authentication.

A Running the system locally for a demo

Commands are run from the repository root. Nothing here contacts Google Cloud.

```
cp .env.example .env            # then put a long random JWT_SECRET (>= 32 bytes) in .
    env
docker compose up -d --build    # whole stack; migrations and provisioning run first
docker compose ps               # health status of each container
```

Seed the supplier directory once, after the databases are migrated. Seeded suppliers reach order-service on the next `supplier-ticker` sweep, within about a minute:

```
docker compose --profile tools build supplier-db-tools
docker compose --profile tools run --rm supplier-db-tools npm run db:seed -- --dry-run
docker compose --profile tools run --rm supplier-db-tools npm run db:seed
```

Then open `http://127.0.0.1:3001`. The first administrator needs `BOOTSTRAP_SECRET` (at least 32 bytes) in `.env` and the `/setup/admin` page.

Watching messages move during a demo:

```
docker compose logs -f saga-orchestrator order-service credit-service credit-publisher
docker compose exec order-db psql -U order -d orders \
  -c "select sequence, type, state, published_at is not null as published from
    order_events order by created_at desc limit 10;"
docker compose exec saga-db psql -U saga -d sagas -c "select order_id, state,
    failure_code from sagas;"
docker compose exec credit-db psql -U credit -d credits \
  -c "select user_id, total_credits, reserved_credits from credit_accounts;"
```

To see the compensation path, stop credit with `docker compose stop credit-service`, create an order, and wait more than 60 seconds plus one `saga-ticker` cycle. The order becomes `ABORTED`. Start credit again and watch the queued commands drain. Hot reloading: `docker compose -f compose.yaml -f compose.dev.yaml up --watch`. Clean slate, which deletes all local data: `docker compose down -v`.

B Reference: snapshot and provenance

Pinned snapshot. Branch `main` of the repository at `/home/hermes/projects/cs3219-p32`, commit `ea16eec5baddbcbddd9412eeb219f563a81b8499` ("Merge pull request #21 from Y2627S1-CS3219-P32/supplier-pubsub"), equal to `origin/main` when inspected, with a clean working tree. Line numbers below refer to that commit. A claim-by-claim map is kept in `docs/service-architecture-guide/CLAIM_MAP.md`.

Programs, ports, databases, startup (Sections 3, 15)
`compose.yaml` 4–544 (services), 545–551 (volumes); healthchecks at 14–18, 56–61, 73–78, 100–109, 136–145, 162–169, 253–262, 312–315, 327–331, 346–350, 376–385, 405–409, 435–444, 481–485, 509–518; tickers at 273–284, 452–466, 529–543; cross-service `depends_on` at 110–122, 263–269, 230–236, 519–526; `.env.example`.

HTTP and auth (Section 4)
`frontend-service/server/api/user-service/login.post.ts` 21–26; `frontend-service/server/services/order-service/client.ts` 7–31; `frontend-service/server/services/credit-service/client.ts` 19–40; `frontend-service/server/api/notification-service/ticket.post.ts`; `frontend-service/nuxt.config.ts` 11–22; `user-service/src/auth.ts` 21–76; `order-service/src/auth.ts` 13–104; `supplier-service/src/auth.ts` 37–100; `credit-service/src/auth.ts` 12–95; `notification-service/src/server.ts` 24–97, `auth.ts` 52–85.

user-events (Sections 5, 8)
`user-service/src/events/user-events.ts` 14–89; `user-service/src/services/auth.service.ts` 43–89, 203, 207–279; `user-service/src/services/users.service.ts` 130–186; `user-service/src/repositories/users.repository.ts` 54–85; `user-service/src/db/schema.ts` 19–42; `user-service/src/worker.ts` 26–147; `supplier-service/src/user-events.ts` 33–67; `order-service/src/events/consumers.ts` 40–75; `credit-service/src/credit/accounts.ts` 63–183.

supplier-events (Sections 7, 11)
`supplier-service/src/supplier-events.ts` 13–182; `supplier-service/src/index.ts` 128–186; `supplier-service/src/reference-routes.ts` 65–97; `supplier-service/src/db/seed.ts` 87–90; `supplier-service/src/worker.ts` 44–93; `supplier-service/src/db/schema.ts` 92–135; `supplier-service/drizzle/0002_supplier_events.sql`; `order-service/src/events/consumers.ts` 77–124; `order-service/src/db/schema.ts` 70–96.

order-events and order-commands (Sections 5, 9)
`order-service/src/orders/write.ts` 27–240; `order-service/src/events/outbox.ts` 16–93; `order-service/src/events/pubsub.ts` 12–92; `order-service/src/app.ts` 40–118; `order-service/src/config.ts` 23–44; `order-service/src/db/schema.ts` 98–159; `order-service/src/index.ts` 11–19.

Saga (Sections 9, 10)
`saga-orchestrator/src/saga.ts` 39–204; `messages.ts` 11–56; `pubsub.ts` 54–77; `outbox.ts` 13–59; `app.ts` 22–58; `config.ts` 18–36; `db/schema.ts` 10–71.

Credit (Sections 5, 9, 10)
`credit-service/src/app.ts` 26–87; `events/provision.ts` 4–83; `events/publisher.ts` 18–116; `events/outbox.ts` 7–42; `events/commands.ts` 25–71; `credit/commands.ts` 24–276; `credit/queries.ts` 41–98; `config.ts` 2–24; `migrations/0000_credit.sql` 60–63; `events/publisher-health.ts`.

Notification (Sections 1, 6)
`notification-service/src/subscription.ts` 16–93; `rules.ts` 23–83; `order-events.ts` 11–30; `store.ts`; `db/schema.ts` 9–33; `index.ts` 14–35.

Cloud transport (Sections 12, 18)
`deploy.sh` 47–80 (topics, push identity), 89–110 (database URLs), 112–155 (setup), 181–206 (subscriptions, scheduler), 209–236 (service resources), 238–316 (event services), 318–420 (deploy); `cloudbuild.yaml` 9–146.

Containers (Sections 14–17)
`order-service/Dockerfile` 3–31 (same layout in `supplier-service`, `notification-service`, `saga-orchestrator`); `credit-service/Dockerfile` 1–17; `user-service/Dockerfile` 3–16; `frontend-service/Dockerfile` 3–16; the `.dockerignore` files; `compose.dev.yaml` 6–132; every `package.json` `scripts` block; `user-service/drizzle.config.ts`; `credit-service/src/db/migrate.ts` 7–24.

Evidence of earlier runs (Section 12)
`AI_USAGE_LOG.md`: “Supplier events (transactional outbox)”, “Saga orchestrator”, “Notification service and live order updates” and “Deploying the order, notification, credit and saga services” entries.